

COMP.2560: System Programming

LAB #8: Pipes

LAB 8: Overview

In class, we discussed the following 4 rules when using Pipes.

Review
Unnamed pipes

The pipe () system call
Rules
Example
The dup2 () system call
Examples

Rules when reading from and writing to a pipe

Rules

When reading from or writing to a pipe, the following rules apply:

1. If a process reads from a pipe whose write-end has been closed, after all data has been read, `read()` returns 0.
2. If a process reads from an empty pipe whose write-end is still open, it sleeps until some input becomes available.
3. If a process tries to read from a pipe more bytes than are present, `read()` reads all available bytes and returns the number of bytes read.
4. If a process writes to a pipe whose read-end has been closed, the write operation fails and the writer process receives a `SIGPIPE`.

In case of multiple processes writing to the same pipe, a write of up to `PIPE_BUF` bytes is guaranteed to be atomic.
→ data from different writer processes will not be interleaved.

Pipes I

In this lab we will explore each and everyone of them.

There are four parts in this lab, described below.

LAB 8 – Part I (Rule #11)

For Rule #1, please see the attached code, `lab8r1.c`, read the code and run the code to see how Rule #1 is realized in the code.

After reviewing the code, you will realize that no child process such that the child process writes, and the parent process reads is realized. You are required to do so, that is:

1. The parent process creates a child process.
 - a. The child process intends to write to the pipe, so the child process closes its read-end of the pipe.
 - b. The parent process intends to read from the pipe, so the parent process closes its write-end of the pipe.
2. The child process writes the message `msg1` to the pipe and then closes its write-end of the pipe.
3. The parent process:
 - a. Reads message `msg1` and displays it, and then

- b. Tries to read again from the pipe and observes the return value of the second read, that should be zero.

Please code the above 3 steps to demonstrate Rule #1 between a parent process and a child process. Make sure you add comments, explaining your work. Submission is required for this question at the end of your lab session.

LAB 8 – Part II (Rule #2)

We have demonstrated Rule #2 in class using code posted on Brightspace.

Indicate what code was used in class to demonstrate this and what changes were indicated to be made in that code so that the behavior of Rule #2 is fully revealed.

No coding is required but you are expected to write a short paragraph to answer the questions above. Submission is required for this question at the end of your lab session.

LAB 8 – Part III (Rule #3)

To demonstrate Rule #3, you can make small changes to `lab8r1.c` so that your code requests to read 32 bytes from the pipe, but only 16 bytes (`msg1`) so far are available in the pipe to be read.

A sample output of such a program could look like the screenshot below:

```
> ...@delta: ~$ ./lab8r1.c
  I have been requested to read 32 bytes, but there are only 16 bytes in the pipe, so far.
  These are the 16 bytes I have read: hello, world #1
> ...@delta: ~$
```

Please make the changes need in `lab8r1.c` to demonstrate the use of pipe's Rule #3. producing a similar output as the screenshot above. Make sure you add comments, explaining your work. Submission is required for this question at the end of your lab session.

LAB 8 – Part IV (Rule #4)

A question related to Rule #4 will be included in the upcoming assignment.

No work or submission is needed for this part of the lab.

LAB 8 – Submission

Submit the items elements described above.

To formally submit your work for this Lab:

- Submit the code (including extended commenting) and the script files (learned in Lab 2) you created for Parts I, and III.
- Submit the paragraph you prepared for Part II.

When submitting, please indicate in the submission textbox the Lab section you belong to.

Evaluation:

1. It is our intention to evaluate your work being done in-person, while you are in the lab.
2. It is strongly suggested that you attend in-person to familiarize yourselves with the Lab infrastructure, meet with the GATAs and solve problems.
3. Each session is only 80 minutes long, so it is strongly suggested that you arrive early, are focused and do not disrupt the process.
4. It is desired that you complete your work while in the lab and get graded after live review from the GATAs.
5. Should you need more time, you can take your work home and present it to the GATAs on the very next lab session (the week following).
6. Even if you decide to work from home and submit your work remotely, you still need to have your work peer reviewed, in-person.